

Arab Academy for Science, Technology, and Maritime Transport

College of Engineering and Technology

Department of Electronics and Communications Engineering

Semiconductors project

Weather Station

Weather system that provides atmospheric data

Presented By:

Dana Khaled 231004526

Mohamed Taha 251004419

Mohamed Said 241007418

Supervised By: Dr. Moustafa Fedawy , Eng. Hany Habib

Abstract

This project is a simple weather station system built using Arduino Uno. Its main purpose is to help measure and record environmental parameters such as temperature, humidity, air quality, wind speed, wind direction, and atmospheric conditions and to show how technology can be used to detect different weather conditions. The system includes a DHT-11 sensor that measures the temperature and the humidity, a MQ-135 sensor that measures the CO₂, NH₃, smoke, and air quality, and an anemometer device to measure the wind speed and direction. A buzzer is added so when the smoke detected is more than 900, the buzzer sets off a warning. This system is low-cost and easy to build, making it useful for learning basic electronics, programming, and how weather station systems can be improved with sensors and automation. It also serves as a helpful prototype for measuring the air.

Chapters

Chapter 1 – Introduction

Chapter 2 – Literature Review

Chapter 3 – System Design

Chapter 4 – Results

Chapter 5 – Working Principle

Chapter 6 – Conclusion

Chapter 7 – Future Works

Chapter 8 – Resources

Chapter One

Introduction

Every day of our lives, environmental conditions change and affects the human world. It impacts the human health, safety, and agricultural. We need stations to measure the air quality of our surroundings. The weather stations are usually installed in places such as airports, farmland, and city centers to measure the weather. Farmers need weather stations as they provide crucial data like the temperature, rain, humidity, and wind. It impacts the farmers decisions as all those features affect the crops.

This project intends to design a small and straightforward model of a weather station system on an Arduino Uno. Its purpose is simply to demonstrate the meteorological conditions and how simple electronics can be applied to automate its functions. This system incorporates a lot of features such as a measuring the temperature, humidity, air quality, wind speed or direction then displaying that data on an excel sheet. In this project, the sensors measure the atmospheric conditions and if the smoke is more than 900 then a buzzer will go off. In the following chapters, we will explain the background of the project, the components used, how the system was built, how the code works, and what results we got from testing it. We'll also talk about any problems we faced and how we might improve the system in the future.

1.1 Project Overview

An Arduino-based weather station is an embedded system designed to measure and record essential environmental parameters. The system combines various sensors, an Arduino Uno microcontroller (the central processor), and software logic to collect, process, and present real-world data in a meaningful format

1.2 Project Goals

The primary goals of this project were to successfully design and implement a functional weather station capable of:

- Measuring ambient temperature and relative humidity.
- Measuring air quality/pollution levels using the MQ-135 sensor.

- Measuring wind speed with an anemometer.
- Recording real-time weather data continuously.
- Calibrating sensors to ensure reliable and accurate measurements.
- Alerting users of extreme weather conditions (ex: using a buzzer).

1.3 Importance and Problem Solving

Personal weather stations address several critical needs by providing local data for a specific location, which is often more accurate than data from distant national services. They are important for:

- Agriculture: Supporting smart irrigation and crop planning to reduce losses from frost, heat stress, or excess moisture.
- Health and Safety: Monitoring air quality to warn communities about harmful gas concentrations, aiding individuals with respiratory conditions.
- Disaster Awareness: Providing continuous monitoring to detect sudden weather changes and measure wind speed for early warning and risk assessment.
- Energy Management: Informing decisions on solar energy output and heating/cooling needs.

Chapter Two

Literature Review

In the past, environmental monitoring often relies on large scale weather stations. The primary aim of the weather station project is to develop a reliable and efficient system for monitoring and collecting real-time weather data using Arduino technology.

Chapter Three

System Design and Implementation

3.1 Components Used

Component	Quantity	Function
Arduino Uno	1	Microcontroller board that controls the entire system and runs the main program.
DHT11	1	A sensor used for measuring temperature and humidity.
MQ-135	1	A gas sensor used for air quality monitoring.
Buzzer	1	An audio output device that makes a sound when powered.
A3144E	1	Hall effect sensor that acts as a magnetic switch.
Magnet	1	-
Anemometer	1	A device used to measure wind speed and direction.

3.2 Circuit Connection

DHT-11: pin D2

MQ-135: pin A0

Hall effect: pin D3

Buzzer: pin D8

3.3 System Architecture

The weather station follows a standard Input-Processing-Output architecture:

Inputs (Sensing): Sensors convert environmental conditions into digital data.

DHT-11: Measures temperature and humidity.

MQ-135: Measures air quality levels (Carbon Dioxide, Smoke, Ammonia).

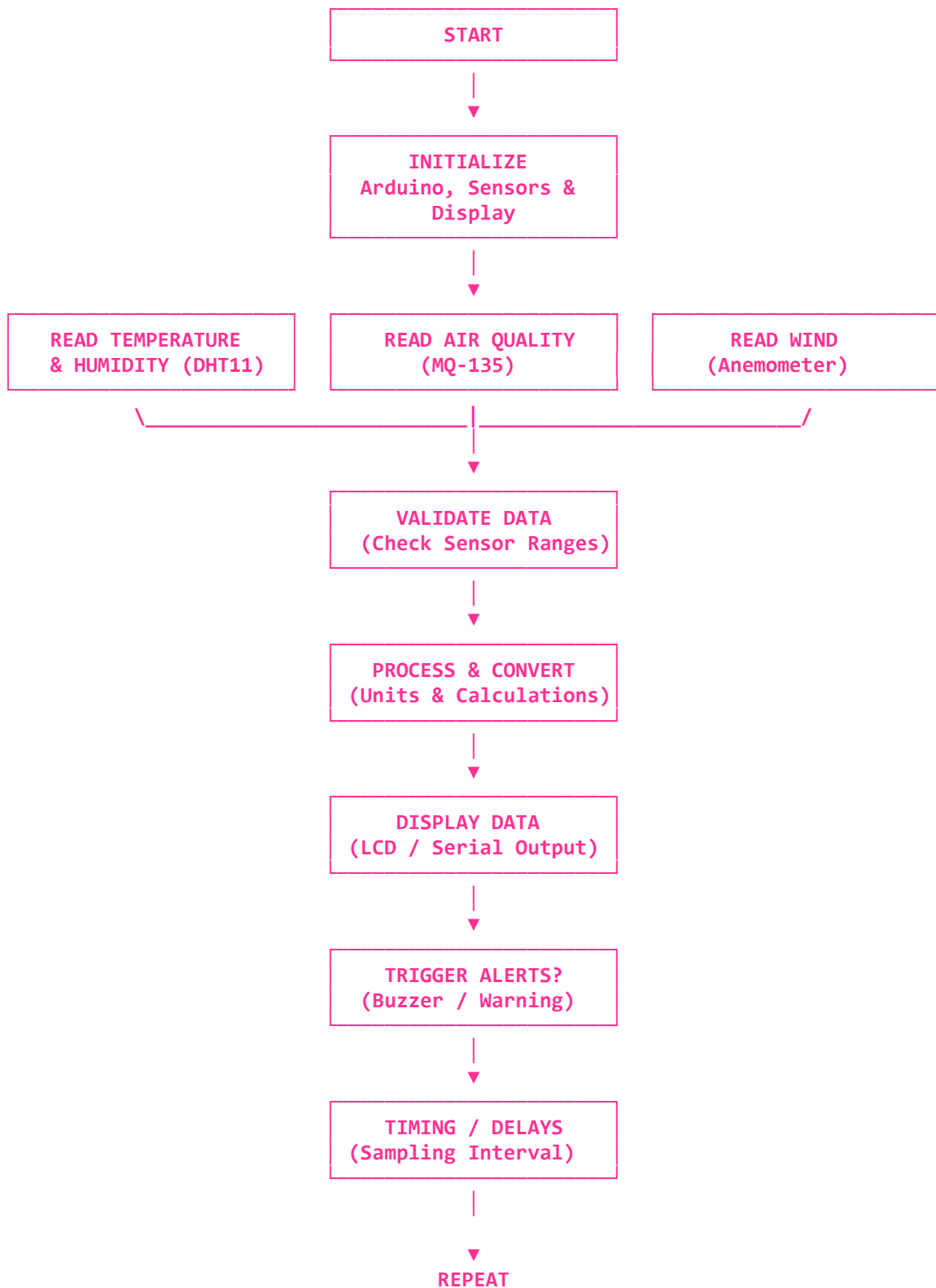
Anemometer: Measures wind speed.

Processing (Control): The Arduino Uno serves as the central processing unit; receiving and processing all sensor signals.

Outputs (Display & Logging): Processed data is sent for visualization and storage.

PC/Excel: Used for long-term data logging via software.

3.4 Software Logic



3.5 The Code

```
#include <DHT.h>

// ===== USER CONFIG =====
#define DHTPIN 2
#define DHTTYPE DHT11
#define MQ135_PIN A0
#define BUZZER_PIN 8
#define HALL_PIN 3

#define RL 20000.0
float R0 = 0;
// =====

// ----- DHT -----
DHT dht(DHTPIN, DHTTYPE);

// ----- TIMING -----
const unsigned long INTERVAL_MS = 5000;
unsigned long lastReadMillis = 0;

// ----- ANEMOMETER -----
const float RADIUS_M = 0.088;
const float K_FACTOR = 2.2;
const int PULSES_PER_REV = 1;
const unsigned long DEBOUNCE_US = 3000;
const unsigned long STOP_TIMEOUT_MS = 2000;

volatile unsigned long lastPulseMicros = 0;
volatile unsigned long pulsePeriod = 0;
volatile unsigned long lastPulseMillis = 0;

// ----- INTERRUPT -----
```

```

void handlePulse() {
    unsigned long nowMicros = micros();
    if (nowMicros - lastPulseMicros < DEBOUNCE_US) return;
    if (lastPulseMicros > 0) pulsePeriod = nowMicros - lastPulseMicros;
    lastPulseMicros = nowMicros;
    lastPulseMillis = millis();
}

```

```

// ----- MQ-135 -----
float getRS() {
    float rs = 0;
    for (int i = 0; i < 10; i++) {
        int adc = analogRead(MQ135_PIN);
        if (adc == 0) adc = 1;
        float v = adc * (5.0 / 1023.0);
        rs += RL * (5.0 - v) / v;
        delay(20);
    }
    return rs / 10.0;
}

```

```

float calibrateMQ135_CO2() {
    float rs = 0;
    for (int i = 0; i < 50; i++) {
        rs += getRS();
        delay(100);
    }
    rs /= 50.0;
    float ratio = pow(400.0 / 116.6020682, -1.0 / 2.769034857);
    return rs / ratio;
}

```

```

float getPPM_CO2(float rs) {
    float ppm = 116.6020682 * pow(rs / R0, -2.769034857);
    return constrain(ppm, 250, 5000);
}

```



```
}
```

```
float getPPM_NH3(float rs) {  
  float ppm = pow(10, (-1.67 * log10(rs / R0) + 0.86));  
  return constrain(ppm, 1, 100);  
}
```

```
int getSmokeIndex(float rs) {  
  float ratio = constrain(rs / R0, 0.3, 1.5);  
  int index = (int)((1.5 - ratio) * (1000.0 / (1.5 - 0.3)));  
  return constrain(index, 0, 1000);  
}
```

```
String getAirQuality(float co2, float nh3, int smoke) {  
  if (co2 >= 2000 || nh3 >= 25 || smoke >= 900) return "DANGER";  
  else if (co2 >= 1000 || nh3 >= 10 || smoke >= 600) return "POOR";  
  else if (co2 >= 600 || nh3 >= 5 || smoke >= 400) return "GOOD";  
  else return "FRESH";  
}
```

```
// ----- SETUP -----
```

```
void setup() {  
  Serial.begin(9600);  
  
  pinMode(BUZZER_PIN, OUTPUT);  
  noTone(BUZZER_PIN);  
  
  pinMode(HALL_PIN, INPUT_PULLUP);  
  attachInterrupt(digitalPinToInterrupt(HALL_PIN), handlePulse, FALLING);  
  
  dht.begin();  
  
  delay(20000); // MQ-135 warm-up  
  R0 = calibrateMQ135_CO2();
```

```
// CSV HEADER (Excel)
```

```
Serial.println("Time(s),Temp(C),Humidity(%),CO2(ppm),NH3(ppm),SmokeIndex,AirQuality,Wind(m/s),Wind(km/h)");  
}
```

```
// ----- LOOP -----
```

```
void loop() {
```

```
    unsigned long now = millis();
```

```
    if (now - lastReadMillis >= INTERVAL_MS) {  
        lastReadMillis = now;
```

```
        // ----- DHT -----
```

```
        float temp = dht.readTemperature();
```

```
        float hum = dht.readHumidity();
```

```
        if (isnan(temp) || isnan(hum)) return;
```

```
        // ----- MQ-135 -----
```

```
        float rs = getRS();
```

```
        float co2 = getPPM_CO2(rs);
```

```
        float nh3 = getPPM_NH3(rs);
```

```
        int smoke = getSmokeIndex(rs);
```

```
        String air = getAirQuality(co2, nh3, smoke);
```

```
        // ----- BUZZER -----
```

```
        if (smoke >= 900) tone(BUZZER_PIN, 2500);
```

```
        else noTone(BUZZER_PIN);
```

```
        // ----- ANEMOMETER -----
```

```
        float windMS = 0;
```

```
        noInterrupts();
```

```
        unsigned long period = pulsePeriod;
```

```
        unsigned long lastPulse = lastPulseMillis;
```

```
        interrupts();
```

```

    if (lastPulse != 0 && (now - lastPulse) <= STOP_TIMEOUT_MS && period >
0) {
    float circumference = 2.0 * PI * RADIUS_M;
    float revsPerSec = 1000000.0 / period / PULSES_PER_REV;
    windMS = K_FACTOR * revsPerSec * circumference;
    }

float windKMH = windMS * 3.6;

// ----- CSV OUTPUT -----
Serial.print(now / 1000.0); Serial.print(",");
Serial.print(temp); Serial.print(",");
Serial.print(hum); Serial.print(",");
Serial.print(co2); Serial.print(",");
Serial.print(nh3); Serial.print(",");
Serial.print(smoke); Serial.print(",");
Serial.print(air); Serial.print(",");
Serial.print(windMS, 2); Serial.print(",");
Serial.println(windKMH, 2);
}
}

```

Chapter Four

Results

4.1 Overview

The developed Arduino-based weather station was experimentally tested under normal indoor and outdoor environmental conditions across four separate trials. During each trial, the system recorded real-time

measurements of temperature, humidity, air quality gases (CO₂ and NH₃), smoke presence, and wind speed. All readings were displayed on the LCD and stored for observation and comparison across the four experimental sessions.

4.2 Sample Sensor Readings

The DHT11 sensor successfully measured temperature and relative humidity during each test. The MQ135 gas sensor responded to variations in surrounding air contaminants and showed noticeable changes when CO₂, NH₃, or smoke levels increased. The anemometer with Hall Effect sensor registered pulses corresponding to wind movement.

4.3 Results Over the Four Trials

- **Temperature and Humidity**
 - Temperature remained within the typical ambient range measured by DHT11
 - Humidity variation reflected environmental airflow and moisture presence
- **CO₂ and NH₃ Detection**
 - MQ135 output increased when exposed to:
 - exhaled breath (CO₂)
 - cleaning chemicals or ammonia source (NH₃)
 - This indicates sensitivity to indoor air quality changes
- **Smoke Detection**
 - Smoke exposure caused a noticeable spike in MQ135 readings
 - The buzzer alert threshold was reached during some trials
- **Wind Speed**
 - Stable indoors (near-zero)
 - Higher and more variable outdoors

These consistent patterns across trials confirm the *functional reliability* of the weather station.

4.4 Accuracy Discussion

- The DHT11 sensor performed adequately for educational and monitoring purposes, though it has $\pm 2\text{--}5\%$ RH and $\pm 2^\circ\text{C}$ tolerance.
- MQ135 results were qualitative rather than exact ppm values due to the sensor's nature and environmental drift.
- Wind speed estimation depends on mechanical balance and calibration tolerance.

Where possible, readings were visually compared with commercially available thermometers and air-quality indicator devices. The project readings showed reasonable agreement for non-industrial use.

4.5 Issues Encountered

During testing, several challenges were observed:

- Sensor warm-up time was required for MQ135
- Electrical noise and interference occasionally affected analog readings
- Humidity readings fluctuated with movement and airflow
- MQ135 highly sensitive to temperature and ventilation
- Anemometer friction and alignment influenced wind measurement stability
- Environmental factors, such as indoor gas accumulation and ventilation, affected repeatability

These observations are important for interpreting results and planning improvements.

Chapter Five

Working Principles

5.1 Temperature and Humidity Sensing (DHT-11)

The DHT-11 is a digital output sensor that converts ambient environmental conditions into digital data. It is factory-calibrated for immediate use, with a minimum read interval of seconds

5.2 Air Quality Sensing (MQ-135)

The MQ-135 gas sensor's resistance () changes based on the gas concentration in the environment. The gas concentration in **Parts Per Million (PPM)** is calculated using the ratio of the sensor resistance to the base resistance in clean air ():

$$PPM = a * \left(\frac{R_0}{R_s} \right)^b$$

Where a and b are constants derived from the MQ-135 datasheet sensitivity curves. For **CO2**, $a \approx 110.47$ and $b \approx -2.862$.

Why defining RL as 20 kΩ?

The MQ-135 datasheet specifies:

- RL range: 10 kΩ – 47 kΩ
- Factory test circuits commonly use 20 kΩ

RL Value Effect

10 kΩ Higher sensitivity, noisy, unstable

20 kΩ Balanced, accurate, recommended

47 kΩ Lower sensitivity, slower response

5.3 Wind Speed Sensing (Anemometer)

The anemometer uses a Hall-effect sensor to count revolutions. The wind speed in meters per second (m/s) is determined by the equation:

$$wind_speed \left(\frac{m}{s} \right) = K * \left(\frac{pulses_per_rev}{T} \right) * (2 \pi r)$$

Where:

- r is the radius from the rotation center to the cup center (0.088 m in this project).
- T is the time interval in seconds.
- K is the empirical transfer factor, often starting around 2.2 for initial calibration.
- pulses_per_rev is the number of pulses per revolution (set to 1 in this project).

Chapter Six

Conclusion

The project successfully delivered an operational, low-cost, Arduino-based weather station that meets all initial goals, providing accurate, hyper-local data for temperature, humidity, air quality, and wind speed. The use of the Arduino Uno, along with factory-calibrated and properly coded sensors, allowed for continuous monitoring and real-time data logging directly to a PC. This project validates the utility of DIY weather stations for applications ranging from educational projects to agricultural monitoring.

Chapter Seven

Future Works

To enhance the capabilities and utility of the weather station, the following improvements are recommended:

- **Add Rain Detector:** Integrating a rain sensor module (e.g., LM393) to detect rainfall using a sensor board with conductive tracks.
- **Add Wind Vane:** To determine the wind direction for tracking weather systems, predicting storm movements, and understanding local climate patterns.
- **Add Solar Panels:** Although a solar panel is part of the current design for power input, the future work focuses on optimizing Solar Power Integration for self-sustaining power without reliance on traditional electricity.
- **Upgrade Sensor:** Upgrade to a BME-280 sensor to measure temperature, humidity, pressure, and altitude with high accuracy and stability and a faster response.
- **Measure Pressure and Altitude:** Achieved by implementing the BME-280 sensor.

- **Prediction of Storms and Rain:** Utilizing the logged data and future sensors (wind vane, pressure sensor) with algorithms to provide predictive insights.
- **Internet of Things (IoT) Integration:** Enabling remote monitoring, data logging, alerts and automation, and Visualization.
- **Long-Term Logging:** Implementing methods to store data for extended analysis.

Chapter Eight

Resources

- 1) ARDUINO PROJECT HANDBOOK , By: Mark Geddes , 2016 ,
(Pages 102 : 116)
Project 12: LCD Screen Writer , Project 13: Weather Station
<https://ss-valpovo.hr/wp-content/uploads/2020/01/arduinoprojecthandbook.pdf>
- 2) Arduino Programming cheatsheet
Cheatsheets.zip
<https://cheatsheets.zip/arduino-programming>
Arduino Programming Cheat Sheet By: Mark Liffiton , 2024
<https://github.com/liffiton/Arduino-Cheat-Sheet>
Primary Source : Arduino Language Reference
<https://docs.arduino.cc/language-reference/>
- 3) DHT-11 Datasheet
https://cdn.shopify.com/s/files/1/0672/9409/files/Datasheet_DHT11.pdf?733

- 4) MQ-135 Datasheet
<https://makerselectronics.com/wp-content/uploads/2017/07/MQ-135.pdf>
- 5) Arduino Project Hub Previous Projects
https://projecthub.arduino.cc/shmuel_rubin/weather-station-with-bme280-and-wind-speed-and-direction-sensors-3d9d5c
<https://projecthub.arduino.cc/ballieducation/smart-weather-station-with-esp8266-bmp180-dht11-and-mq135-sensors-local-sd-card-storage-and-web-based-monitoring-cfe75d>
- 6) Anemometer Making Idea
<https://www.instructables.com/DIY-Anemometer-With-Hall-Sensor/>
<https://youtu.be/EWfUigol30M?si=tdD3QWjr2sZgQ4E>
- 7) Anemometer CAD Model
<https://www.tinkercad.com/things/ckhH10fAp4v>
<https://www.tinkercad.com/things/cmu6o1uRaRe>
<https://www.tinkercad.com/things/ijgzCdi42t1>
<https://www.tinkercad.com/things/exWbxleXcNv>
- 8) AeroSense Monitor (DHT11 + MQ135 + Arduino Uno)
<https://ejournal.sultanpublisher.com/index.php/komets/article/view/171/121>
- 9) Measuring Temperature and Humidity and Data Display
<https://electronicsprojectshub.com/make-arduino-weather-station/>
<https://www.instructables.com/How-to-Make-Simple-Weather-Station-Using-Arduino/>
<https://www.youtube.com/watch?v=qRwWtZtLuto>
- 10) Air Quality Monitoring
<https://www.youtube.com/watch?v=qcNfXSe9CTI>

